

ArchitectAI

Software Architecture Blueprint

make an prototype of ticket app

Generated: [25/6/2026](#)
Blueprint ID: [0822bdfc](#)
AI Model: [gemini-2.5-flash](#)
Generation Time: [222.3s](#)
Source: [AI Generated](#)
Blueprint Score: [85 / 100](#)

1. Executive Summary

Value Proposition: A streamlined mobile application that simplifies event creation and ticket sales for organizers, while offering a seamless ticket purchasing and management experience for attendees. It aims to reduce administrative overhead and enhance user satisfaction.

Project Type: Mobile App

Industry: Event Management

Estimated Complexity: medium

Development Timeline: 4-6 months

Suggested Team Size: 1 PM, 2 Backend Devs, 2 Frontend/Mobile Devs, 1 UI/UX

2. Problem Statement

Inefficient manual ticketing processes, lack of real-time insights for event organizers, and fragmented attendee experiences often lead to frustration. This app solves these by providing a centralized, intuitive platform for digital ticketing and event management.

3. Key Requirements

Functional Requirements

- [must-have] FR-001: User Registration and Login
- [must-have] FR-002: Event Creation and Publishing
- [must-have] FR-003: Ticket Type Configuration
- [must-have] FR-004: Secure Ticket Purchase Flow
- [must-have] FR-005: Digital Ticket Display (QR/Barcode)
- [must-have] FR-006: Ticket Scanning and Validation

Non-Functional Requirements

- NFR-001: Fast Ticket Purchase Process — Page load time < 2s; Transaction completion < 5s
- NFR-002: Secure Payment Processing — PCI DSS Level 1 compliance, TLS 1.2+ encryption
- NFR-003: High Concurrency Support — 99.9% uptime during peak load, response time within 3 seconds for 10,000 concurrent users
- NFR-004: Intuitive User Interface — System Usability Scale (SUS) score > 80, Task completion rate > 95%

4. Core Features

CF-001: User Authentication & Profiles

Allows users to create accounts, log in, manage their personal information, and set preferences.

CF-002: Event Discovery & Search

Enables attendees to browse, search, and filter events based on various criteria like category, date, location.

CF-003: Event Creation & Management for Organizers

Provides a dashboard and tools for organizers to create, edit, and publish their events.

CF-004: Secure Ticket Purchase & Payment

Facilitates a seamless and secure process for attendees to select and purchase tickets.

CF-005: Digital Ticket Wallet & Display

Allows attendees to access and view their purchased tickets with unique scannable QR/barcodes.

CF-006: Ticket Scanning & Validation (Entry Management)

Provides a dedicated scanning tool for event staff to validate tickets at the event entrance.

5. Technology Stack

6. Database Design

Database Type: PostgreSQL

Scaling Strategy: Database scaling will involve horizontal partitioning (sharding) for high-traffic tables like `Tickets` and `Orders` if necessary, read replicas for scaling read operations, and connection pooling for efficient resource utilization. Caching with Redis will offload frequent read requests from the database.

A relational database will store structured data like user profiles, event details, ticket information, and orders. PostgreSQL is chosen for its robustness, ACID compliance, and strong support for complex queries and transactional integrity, critical for a ticketing system. It will be complemented by Redis for caching frequently accessed data to improve performance, especially during peak ticket sales.

Collections / Tables

Users:

- id (UUID) *
- email (String) *
- passwordHash (String) *
- firstName (String) *

- lastName (String) *

Events:

- id (UUID) *
- organizerId (Relation) *
- title (String) *
- description (String) *
- startDate (DateTime) *

EventVenues:

- id (UUID) *
- name (String) *
- address (String) *
- city (String) *
- country (String) *

TicketTypes:

- id (UUID) *
- eventId (Relation) *
- name (String) *
- price (Float) *
- quantityAvailable (Int) *

7. System Architecture

The system will follow a client-server architecture. The mobile application, built with React Native, will serve as the frontend, communicating with a RESTful API backend built on Node.js/Express. PostgreSQL will serve as the primary persistent data store, with Redis used for caching to enhance performance. File storage for images (e.g., event banners) will leverage AWS S3. Third-party integrations will handle payments (Stripe), email (SendGrid), and location services (Google Maps API).

Frontend: React Native

Backend: Node.js/Express (with TypeScript)

Database: PostgreSQL

Deployment: AWS (EC2, RDS, S3, Lambda)

8. Development Roadmap

Total Duration: 4-6 months

Team Size: 1 PM, 2 Backend Devs, 2 Frontend/Mobile Devs, 1 UI/UX

Phase 1: Foundation & Core Backend (1 month)

Goal: Establish core infrastructure, user authentication, and basic event creation/listing.

- Cloud infrastructure setup (AWS accounts, VPC, basic services)
- Backend API boilerplate (Node.js/Express, TypeScript)
- PostgreSQL database schema for Users, Events, Venues, TicketTypes
- User Registration and Login (JWT)
- Event Creation (draft status) and basic listing API
- Mobile app basic navigation and authentication screens

Phase 2: Ticket Lifecycle & Attendee Experience (1.5 months)

Goal: Implement secure ticket purchase flow, digital ticket display, and event discovery features.

- TicketType management for organizers (prices, quantities, sales periods)
- Secure payment gateway integration (Stripe)
- Order and Ticket generation upon successful payment
- Digital Ticket Wallet with QR/Barcode generation
- Event Search & Filtering for attendees
- User profile and order history views
- NFRs for performance (purchase speed) and security (PCI DSS)

Phase 3: Organizer & Staff Tools + Admin (1.5 months)

Goal: Build event management features for organizers, ticket scanning for staff, and initial admin dashboard.

- Event publishing workflow (including admin approval)
- Organizer dashboard with basic sales overview
- Ticket Scanning & Validation application (separate or integrated into main app for staff role)
- Admin dashboard for event moderation and user management
- Notifications module (organizer to attendees)
- File storage integration (AWS S3) for event images

Phase 4: Refinement, Testing & Launch Preparation (1 month)

Goal: Address non-functional requirements, conduct comprehensive testing, and prepare for public launch.

- Full system integration testing
- Performance and load testing (NFR-001, NFR-003)
- Security audit and penetration testing (NFR-002)
- UI/UX polishing and accessibility checks (NFR-004)
- Bug fixing and stabilization
- Deployment to production environment
- Monitoring and alerting setup

9. Security Considerations

Authentication: JWT (JSON Web Tokens) with robust refresh token mechanisms. Social logins (Google/Apple) integration.

Authorization: Role-Based Access Control (RBAC) middleware for API endpoints (attendee, organizer, admin, staff roles).

- End-to-end encryption (TLS 1.2+) for all communications
- Hashing and salting of all passwords (bcrypt)
- Data encryption at rest for sensitive data (PostgreSQL TDE or AWS RDS encryption)
- Input validation and sanitization to prevent injection attacks
- Regular security audits and penetration testing

10. AI Quality Review — Score: 85/100

Strengths

- Clear definition of user roles and their associated functionalities, leading to well-scoped features.
- Robust relational database design (PostgreSQL) with strong ACID properties, ideal for financial transactions.
- Comprehensive API endpoint definitions, including authentication, rate limiting, and detailed schemas.
- Strategic use of React Native for cross-platform mobile development, leveraging a single codebase.
- Consideration for key non-functional requirements like security, scalability, and performance from the outset.
- Structured roadmap with clear phases, goals, and deliverables, aligning with agile methodologies.
- Inclusion of admin features and basic AI considerations for future growth.

Weaknesses

- The mobile app's folder structure is somewhat generic; specific React Native best practices (e.g., Atomic Design or specific feature-based organization) could be more detailed.
- AI features are outlined but lack deep architectural integration details in the initial phases, relying on future expansion which could introduce scope creep if not managed.
- The `staff` role's access and capabilities, especially regarding device security for ticket scanning, could be more explicitly defined (e.g., dedicated app vs. in-app role switching).
- Manual ticket lookup for staff is mentioned, but the UI/UX for handling exceptions (e.g., ticket not scanning) is not detailed.
- The blueprint heavily relies on third-party integrations (Stripe, Google Maps, SendGrid), which introduces external dependencies and potential vendor lock-in risks.
- No explicit mention of robust logging and tracing for distributed systems, which is crucial for troubleshooting in production.

Suggested Improvements

- Elaborate on the mobile app's architecture and state management strategy, specifying patterns like Redux Toolkit or React Context API with use-cases.
- Integrate a robust logging and monitoring solution (e.g., ELK stack, Datadog) early in the DevOps plan.
- Define clear API versioning strategy to accommodate future changes without breaking existing clients.
- For AI features, consider a 'minimum viable AI' approach in earlier phases, perhaps starting with basic rule-based recommendations before evolving to machine learning models.
- Conduct a thorough third-party vendor assessment to mitigate risks associated with external integrations.
- Develop a clear communication plan for event organizers and attendees, including how announcements and updates will be managed via the app and other channels.
- Explore GraphQL for the API layer in future iterations to provide more flexible data fetching for the mobile client.